

Annex: Project Report "Intelligent Restaurant Statistics"

1. Project justification

In this annex I describe my project **"Intelligent Restaurant Statistics"**, based on the accepted proposal. My goal has been to build a web application that:

- displays restaurant sales statistics,
- receives structured JSON from an API,
- triggers data loading with a "Load data" button,
- displays that data in interactive charts.

The proposal defined the user experience and the architecture I implemented:

- lightweight frontend with **HTML, CSS and JavaScript**.
- serverless backend with **AWS Lambda, API Gateway** and a cloud database.
- static hosting of the interface in **AWS S3**.
- chart rendering with libraries such as **Chart.js**.

This approach matches the current implementation, which includes a static frontend with a load button and a real AWS API.

2. Real AWS deployment and serverless architecture

This is not just a theoretical architecture: the solution has been designed to work with real AWS services.

- The frontend can be hosted as a static website in an **Amazon S3** bucket.
- The data API uses a public **Amazon API Gateway** endpoint.
- The backend logic can run on **AWS Lambda**.
- For a typical serverless design, the recommended data store is **DynamoDB**, although I also justify **RDS MySQL** when an academic requirement calls for a relational database.

Why serverless?

- Less server administration.
- Managed automatic scaling.
- Pay for actual usage, without idle instances.
- Ideal for demos and academic projects where volume does not justify full server infrastructure.

Database justification

- ****DynamoDB**** is the natural choice for serverless: fast access, flexible schema, and automatic scaling.
- ****RDS MySQL**** can be justified if there is an academic requirement to use relational SQL. I explain that this choice responds to that academic requirement, not because it is the ideal option for a pure serverless architecture.

3. Functional requirements (FR) and non-functional requirements (NFR)

Clear, measurable functional requirements

- ****FR1****: When the user presses the ****"Load data"**** button, the application must request data from the endpoint and render the charts.
- ****FR2****: The application must manage the loading state, showing an indicator while it receives the response.
- ****FR3****: It must display an error message if the API returns a failure, the network is unavailable, or the JSON is invalid.
- ****FR4****: It must show at least four different charts: revenue by category, orders by hour, best-selling items, and daily revenue.

Clear, measurable non-functional requirements

- ****NFR1****: Response time for the `/sales`` endpoint should be under 500 ms in a simple demo scenario.
- ****NFR2****: Frontend availability should be at least 99.5% while the S3 bucket and endpoint are accessible.
- ****NFR3****: Estimated monthly cost should be low, below €20-30 for a demo-level usage.
- ****NFR4****: Frontend-backend integration documentation should be available so another developer can understand the flow.

4. Data model: simple but reasoned

The current data model focuses on two main entities:

- ****products****
- ****sales****

Why simple?

This simplification is intentional. For an academic submission it is preferable to:

- keep the scope manageable,
- focus the solution on the data flow and visualization,
- avoid excessive complexity that would make the demo harder to validate.

With these two entities I can still obtain relevant metrics and build a meaningful dashboard.

How the model can scale

The model is scalable and can be expanded with additional entities such as:

- ``restaurants``
- ``employees``
- ``tables``
- ``tickets``
- ``payment_methods``
- ``categories``
- ``customers``

In a multi-restaurant scenario, the correct approach is to add a ``restaurant_id`` field to ``products`` and ``sales``, or to use another multi-tenancy strategy.

5. Frontend-backend integration

Actual integration flow

- The user presses `**"Load data"**`.
- ``main.js`` sends a ``fetch()`` request to ``https://9nccdykio2.execute-api.eu-north-1.amazonaws.com/prod/sales``.
- The HTTP status is validated and the timeout is handled.
- The JSON is parsed, records are extracted, and Chart.js is used.
- The charts update dynamically on the page.

API documentation

The endpoint returns JSON with a ``salesRecords`` array. Each object can include:

- ``date``
- ``totalRevenue``
- ``revenueByCategory``

- ``ordersByHour``
- ``itemsSold``

This format allows the dashboard to render the various charts.

6. Costs

Cost estimate for a demo

- **S3**: very low, almost free for a few megabytes of static assets.
- **API Gateway / Lambda**: pay per invocation and execution time; inexpensive under light usage.
- **DynamoDB**: cheap under low demand when using on-demand mode.
- **RDS MySQL**: more expensive than DynamoDB; it is justified only if there is an academic requirement.

Cost optimization

- use S3 + CloudFront for static frontend hosting,
- limit Lambda execution time and memory footprint,
- use DynamoDB on-demand or autoscaling instead of fixed provisioned capacity,
- if using RDS, select small instances and manage snapshots.

7. Security and exposure of information

`/sales`` endpoint without authentication

I have left the endpoint unauthenticated for this academic demo. This approach makes evaluation and demonstration easier.

However, in a production environment this would be critical because:

- it would allow access to sensitive sales data,
- it would expose the system to abuse and scraping,
- it would prevent control over who can query the information.

Infrastructure exposure

Although the password may appear hidden in documentation, I included a public endpoint and infrastructure details to explain the design.

I justified this approach as part of an academic project, although in a production product it would be advisable to:

- hide database names and internal endpoints,
- use **AWS Secrets Manager** or **Parameter Store** for secrets,
- restrict access by VPC/IP and minimal roles,
- apply strict IAM policies.

8. Definition of "intelligent"

The project is called **Intelligent Restaurant Statistics** even though it does not include predictions or recommendations.

In this context, "intelligent" means:

- dynamic visualizations that transform JSON data into insights,
- an interface that allows users to explore metrics without manual preprocessing,
- a dashboard that highlights comparisons and sales patterns.

It is a **descriptive** dashboard, not a predictive system. I explain that intelligence is understood in terms of visual analysis and ease of use.

9. Scope and limitations: why there is no administration panel

The dashboard loads data from the API, but it does not include a panel to:

- insert sales manually,
- import CSV files,
- connect a POS,
- manage products.

I chose to prioritize:

- data visualization,
- the data flow between frontend and backend,
- validating the serverless architecture.

In this phase I focused on analysis and data presentation; data entry is left for a later extension.

10. Extended SWOT analysis

Strengths

- Real AWS deployment with managed services.
- Clear and reasoned serverless architecture.
- Intuitive interface with a central load button.
- Real REST API consumption and JSON-to-chart transformation.
- Documentation of the flow and endpoint.
- Demonstrable project executable in a real environment.

Weaknesses

- The chosen scope focuses on data visualization and does not yet include predictions, recommendations, or advanced comparisons.
- This version does not include a full interface to register sales, import CSV, or manage products; the focus has been to validate the model and the visualization.
- The `/sales` endpoint operates without authentication in the demo environment, which makes testing straightforward.
- The presentation includes infrastructure detail needed to explain the design, although in a production setting this visibility would be managed more carefully.
- The data model is compact, but it provides a clear foundation for future expansion.

Opportunities

- Extend the project with demand predictions and menu recommendations.
- Add CSV import and POS connectors to make the solution more complete.
- Integrate with real tools such as Google Looker Studio or hospitality SaaS solutions.
- Use the serverless architecture to scale to multiple restaurants.
- Apply ML in a later phase to detect trends and suggest actions.

Threats

- Competition from established POS and SaaS dashboard solutions.
- The need for real data security if the project moves to a commercial environment.
- Cost increases from larger infrastructure requirements as data grows.
- The risk that the project may be perceived as too basic if operational features are not expanded.

11. Answers to key questions

Why does the model only include `products` and `sales`?

I designed it as a dashboard demo. With these two entities I was able to demonstrate the complete flow from data acquisition to metrics visualization. Reducing the model helped keep the prototype functional and clear.

How would you adapt the model for multiple restaurants?

I would add a `restaurants` entity and reference it from `products` and `sales` using a `restaurant\_id` field.

Options:

- logical multi-tenancy: all tables share the same database with `restaurant\_id`.
- physical multi-tenancy: separate databases per restaurant.

I would also add access control so each restaurant only sees its own data.

What happens if a product with associated sales is deleted?

In a relational schema, the recommended approach is not to physically delete the product if it has historical sales. A good practice is to use soft deletion (`is\_active`, `deleted\_at`) or to apply `ON DELETE RESTRICT` in the relationship.

If the product is deleted physically, there is a risk of losing historical integrity and leaving incomplete sales records. Another alternative is to denormalize key product information inside each `sales` record to preserve history.

12. Conclusion

In this annex I summarize the justification of the project, its alignment with the proposal, its limitations, and its strengths. The work is a functional demo focused on sales visualization and AWS architecture. I explain that the chosen approach responds to an academic scope and a first development milestone.